# Sri Lanka Institute of Information Technology

**Relational Schema Design, SQL Implementation, Queries and Advanced SQL Features**

**Web-based Laundry Management System**

Group ID: 2025-Y2-S1-MLB-B7G1-04
Project group - 124
Year 2 Semester 1 - 2025

**Database Design and Development - IT2140**

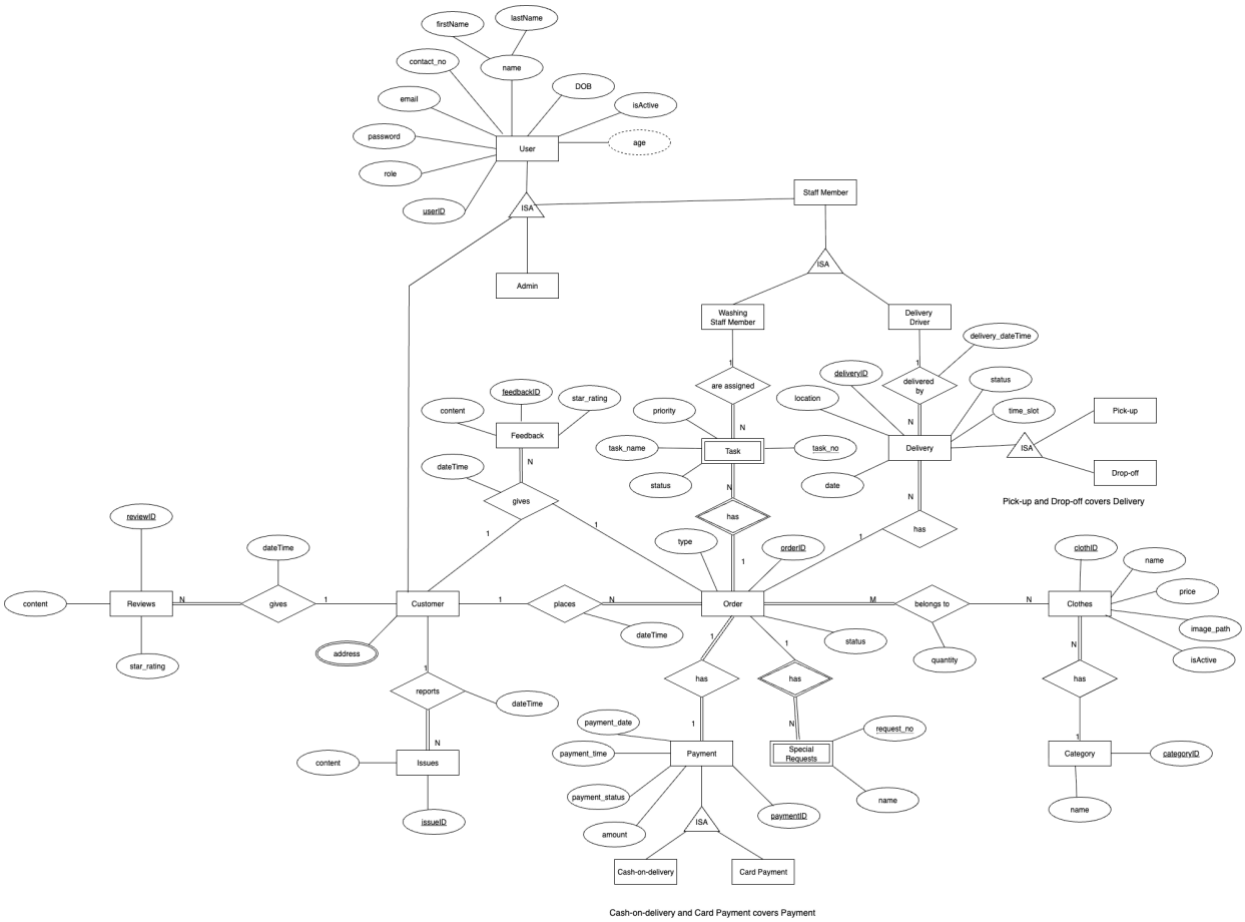
Assignment 01 part - 02

2025/10/12

## Group Members

| | Student ID number | Name |
|---|---|---|
| **1** | IT24101601 | Asma M.F |
| **2** | IT24102180 | Assalaarachchi B.T |
| **3** | IT24101599 | Jayasinghe S.I. |
| **4** | IT24101613 | Kodagoda D.S |
| **5** | IT24300057 | Razeen M.R.M |
| **6** | IT24300059 | Bandarayapa Y.A.R.J |

# Modified EER diagram



# Part A – Mapping EER to Relational Schema

**Order** (**orderID:** INT, **customerID:** INT, **type:** VARCHAR, **order_dateTime:** DATETIME, **status:** VARCHAR)

**User** (**userID:** INT, **firstName:** VARCHAR, **lastName:** VARCHAR, **email:** VARCHAR, **password:** VARCHAR, **DOB:** DATE, **isActive:** BIT, **contact_no:** CHAR(10), **role:** VARCHAR)

**Customer_address** (**customerID:** INT, **address:** VARCHAR)

**Payment** (**paymentID:** INT, **orderID:** INT, **date:** DATE, **time:** TIME, **amount:** DECIMAL, **type:** VARCHAR, **status:** VARCHAR)

**OrderItem** (**orderID:** INT, **clothID:** INT, **quantity:** INT)

**Delivery** (**deliveryID:** INT, **orderID:** INT, **location:** VARCHAR, **date:** DATE, **time_slot:** VARCHAR, **delivery_dateTime:** DATETIME, **type:** VARCHAR, **driverID:** INT, **status:** VARCHAR)

**Cloth** (**clothID:** INT, **name:** VARCHAR, **unit_price:** DECIMAL, **categoryID:** INT, **isActive:** BIT, **image_path:** VARCHAR)

**Category** (**categoryID:** INT, **name:** VARCHAR)

**Task** (**orderID:** INT, **task_no:** INT, **name:** VARCHAR, **priority:** VARCHAR, **staffID:** INT, **status:** VARCHAR)

**SpecialRequest** (**orderID:** INT, **request_no**: INT, **name:** VARCHAR)

**Issue** (**issueID:** INT, **customerID:** INT, **content:** VARCHAR, **issue_dateTime:** DATETIME)

**Review** (**reviewID:** INT, **customerID:** INT, **content:** VARCHAR, **star_rating:** INT, **review_dateTime:** DATETIME)

**Feedback** (**feedbackID:** INT, **orderID:** INT, **customerID:** INT, **content:** VARCHAR, **star_rating:** INT, **feedback_dateTime:** DATETIME)

## (i) Constraints applied

**Order table**

**type:** NOT NULL **status:** DEFAULT 'Pending'

**User table**

**firstName:** NOT NULL **lastName:** NOT NULL **email:** UNIQUE, NOT NULL

**password:** UNIQUE, NOT NULL

**contact_no:** should contain only 10 digits starting with 0

**role:** DEFAULT 'CUSTOMER', **isActive:** DEFAULT 1

**Payment table**

**amount:** NOT NULL

**OrderItem table**

**quantity:** NOT NULL

**Delivery table**

**location:** NOT NULL **type:** NOT NULL **status:** DEFAULT 'Processing'

**Cloth table**

**categoryID:** NOT NULL **unit_price:** NOT NULL **isActive:** DEFAULT 1

**Category table**

**name:** NOT NULL

**Task table**

**name:** NOT NULL **status:** DEFAULT 'Not completed'

**SpecialRequest table**

**name:** NOT NULL

**Issue table**

**content:** NOT NULL

**Review table**

**content:** NOT NULL

**Feedback table**

**content:** NOT NULL

## (ii) ISA mappings

Used option 3 for all ISA hierarchies because all subclasses are disjointed and have a few attributes.

# Part B – SQL DDL Implementation

```sql
CREATE DATABASE Laundry_db;
GO

USE Laundry_db;
GO

CREATE TABLE Users (
    userID INT PRIMARY KEY IDENTITY,
    firstName VARCHAR(50) NOT NULL,
    lastName VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(20),
    DOB DATE,
    isActive BIT DEFAULT 1,
    contact_no CHAR(10) CHECK (contact_no LIKE '0[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]'),
    role VARCHAR(50) NOT NULL
);

CREATE TABLE Customer_address (
    customerID INT PRIMARY KEY,
    address VARCHAR(255),
    FOREIGN KEY (customerID) REFERENCES Users(userID)
);

CREATE TABLE Orders (
    orderID INT PRIMARY KEY IDENTITY,
    customerID INT NOT NULL,
    type VARCHAR(50) NOT NULL,
    order_dateTime DATETIME,
    status VARCHAR(50) DEFAULT 'Pending',
    FOREIGN KEY (customerID) REFERENCES Users(userID)
);

CREATE TABLE Payment (
    paymentID INT PRIMARY KEY IDENTITY,
    orderID INT NOT NULL,
    date DATE,
    time TIME,
    amount DECIMAL(10,2) NOT NULL,
    type VARCHAR(50),
    status VARCHAR(50),
    FOREIGN KEY (orderID) REFERENCES Orders(orderID)
);
```

```sql
CREATE TABLE Delivery (
    deliveryID INT PRIMARY KEY IDENTITY,
    orderID INT NOT NULL,
    location VARCHAR(255),
    deliveryDate DATE,
    time_slot VARCHAR(100),
    delivery_dateTime DATETIME,
    type VARCHAR(100) NOT NULL,
    driverID INT,
    status VARCHAR(50) DEFAULT 'Processing',
    FOREIGN KEY (orderID) REFERENCES Orders(orderID),
    FOREIGN KEY (driverID) REFERENCES Users(userID)
);

CREATE TABLE Category (
    categoryID INT PRIMARY KEY IDENTITY,
    name VARCHAR(50) NOT NULL
);

CREATE TABLE Cloth (
    clothID INT PRIMARY KEY IDENTITY,
    name VARCHAR(100),
    unit_price DECIMAL(10,2) NOT NULL,
    categoryID INT NOT NULL,
    isActive BIT DEFAULT 1,
    image_path VARCHAR(100),
    FOREIGN KEY (categoryID) REFERENCES Category(categoryID)
);
```

```sql
CREATE TABLE Task (
    orderID INT,
    task_no INT,
    name VARCHAR(100),
    priority VARCHAR(50),
    staffID INT,
    status VARCHAR(50),
    PRIMARY KEY (orderID, task_no),
    FOREIGN KEY (orderID) REFERENCES Orders(orderID)
);

CREATE TABLE OrderItem (
    orderID INT NOT NULL,
    clothID INT NOT NULL,
    quantity INT NOT NULL,
    PRIMARY KEY (orderID, clothID),
    FOREIGN KEY (orderID) REFERENCES Orders(orderID),
    FOREIGN KEY (clothID) REFERENCES Cloth(clothID)
);

CREATE TABLE SpecialRequest (
    request_no INT,
    orderID INT,
    name VARCHAR(100) NOT NULL,
    PRIMARY KEY (request_no, orderID),
    FOREIGN KEY (orderID) REFERENCES Orders(orderID)
);
```

```sql
CREATE TABLE Issue (
    issueID INT PRIMARY KEY IDENTITY,
    customerID INT NOT NULL,
    content VARCHAR(255),
    issue_dateTime DATETIME,
    FOREIGN KEY (customerID) REFERENCES Users(userID)
);

CREATE TABLE Feedback (
    feedbackID INT PRIMARY KEY IDENTITY,
    customerID INT,
    orderID INT,
    content VARCHAR(255) NOT NULL,
    star_rating INT,
    feedback_dateTime DATETIME,
    FOREIGN KEY (orderID) REFERENCES Orders(orderID),
    FOREIGN KEY (customerID) REFERENCES Users(userID)
);

CREATE TABLE Review (
    reviewID INT PRIMARY KEY IDENTITY,
    customerID INT,
    content VARCHAR(255) NOT NULL,
    star_rating INT,
    review_dateTime DATETIME,
    FOREIGN KEY (customerID) REFERENCES Users(userID)
);
```

## Part C – Insert Sample Data

| | userID | firstName | lastName | email | password | DOB | isActive | contact_no | role |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | John | Smith | john@gmail.com | pass123 | 2001-01-15 | 1 | 0771234567 | Customer |
| 2 | 2 | Emma | Brown | emma@gmail.com | pass456 | 2002-03-22 | 1 | 0719876543 | Driver |
| 3 | 3 | Liam | Jones | liam@gmail.com | pass789 | 2004-07-08 | 1 | 0751122334 | Staff |
| 4 | 4 | Olivia | Taylor | olivia@gmail.com | pass321 | 2003-11-30 | 1 | 0789988776 | Customer |
| 5 | 5 | Noah | Davis | noah@gmail.com | pass999 | 2004-05-05 | 1 | 0704455667 | Customer |

| | customerID | address |
|---|---|---|
| 1 | 1 | 123 Main Street |
| 2 | 2 | 321 Temple Road |
| 3 | 3 | 654 Maple Lane |
| 4 | 4 | 456 Oak Avenue |
| 5 | 5 | 789 Temple Road |

| | orderID | customerID | type | order_dateTime | status |
|---|---|---|---|---|---|
| 1 | 1 | 1 | Washing | 2025-10-01 09:00:00.000 | Pending |
| 2 | 2 | 4 | Ironing | 2025-10-02 10:30:00.000 | Pending |
| 3 | 3 | 5 | Dry Cleaning | 2025-10-03 11:45:00.000 | Pending |
| 4 | 4 | 1 | Ironing | 2025-10-04 14:15:00.000 | Pending |
| 5 | 5 | 5 | Washing & Ironing | 2025-10-05 16:00:00.000 | Pending |

| | paymentID | orderID | date | time | amount | type | status |
|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 2025-10-01 | 09:15:00 | 4500.00 | Card | Paid |
| 2 | 2 | 2 | 2025-10-02 | 10:45:00 | 1200.00 | COD | Pending |
| 3 | 3 | 3 | 2025-10-03 | 12:00:00 | 3000.00 | Card | Paid |
| 4 | 4 | 4 | 2025-10-04 | 14:30:00 | 1500.00 | Card | Failed |
| 5 | 5 | 5 | 2025-10-05 | 16:15:00 | 2200.00 | COD | Paid |

| | deliveryID | orderID | location | deliveryDate | time_slot | delivery_dateTime | type | driverID | status |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | Colombo | 2025-10-06 | Morning | 2025-10-06 08:00:00.000 | Standard | 2 | Processing |
| 2 | 2 | 2 | Kandy | 2025-10-06 | Afternoon | 2025-10-06 13:00:00.000 | Express | 2 | Processing |
| 3 | 3 | 3 | Galle | 2025-10-07 | Evening | 2025-10-07 18:00:00.000 | Standard | 2 | Processing |
| 4 | 4 | 4 | Negombo | 2025-10-07 | Morning | 2025-10-07 09:00:00.000 | Express | 2 | Processing |
| 5 | 5 | 5 | Matara | 2025-10-08 | Afternoon | 2025-10-08 14:00:00.000 | Standard | 2 | Processing |

| | categoryID | name |
|---|---|---|
| 1 | 1 | Casual |
| 2 | 2 | Formal |
| 3 | 3 | Sportswear |
| 4 | 4 | Partywear |
| 5 | 5 | Curtains |

| | clothID | name | unit_price | categoryID | isActive | image_path |
|---|---|---|---|---|---|---|
| 1 | 1 | T-Shirt | 1500.00 | 1 | 1 | tshirt.jpg |
| 2 | 2 | Blazer | 4500.00 | 2 | 1 | blazer.jpg |
| 3 | 3 | Track pants | 2000.00 | 3 | 1 | trackpants.jpg |
| 4 | 4 | Saree | 3500.00 | 4 | 1 | saree.jpg |
| 5 | 5 | Curtain | 5000.00 | 5 | 1 | curtain.jpg |

| | orderID | clothID | quantity |
|---|---|---|---|
| 1 | 1 | 1 | 2 |
| 2 | 2 | 2 | 1 |
| 3 | 3 | 3 | 3 |
| 4 | 4 | 4 | 1 |
| 5 | 5 | 5 | 2 |

| | orderID | task_no | name | priority | staffID | status |
|---|---|---|---|---|---|---|
| 1 | 1 | 1 | Wash | High | 3 | Completed |
| 2 | 2 | 1 | Iron | Medium | 3 | Pending |
| 3 | 3 | 1 | Fold | Low | 3 | Completed |
| 4 | 4 | 1 | Pack | High | 3 | Pending |
| 5 | 5 | 1 | Deliver | High | 3 | Scheduled |

| | request_no | orderID | name |
|---|---|---|---|
| 1 | 1 | 1 | Dry clean only |
| 2 | 2 | 2 | Express delivery |
| 3 | 3 | 3 | Include invoice |
| 4 | 4 | 4 | Eco packaging |
| 5 | 5 | 5 | Custom embroidery |

| | issueID | customerID | content | issue_dateTime |
|---|---|---|---|---|
| 1 | 1 | 1 | Late delivery | 2025-10-06 17:00:00.000 |
| 2 | 2 | 4 | Wrong item | 2025-10-06 18:30:00.000 |
| 3 | 3 | 5 | Delivery was late | 2025-10-07 09:45:00.000 |
| 4 | 4 | 1 | Not cleaned well | 2025-10-07 11:15:00.000 |
| 5 | 5 | 5 | Damaged packaging | 2025-10-08 15:00:00.000 |

| | feedbackID | customerID | orderID | content | star_rating | feedback_dateTime |
|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | Loved the quality. | 5 | 2025-10-06 17:30:00.000 |
| 2 | 2 | 4 | 2 | Could improve delivery time. | 3 | 2025-10-06 19:00:00.000 |
| 3 | 3 | 5 | 3 | Great customer service. | 4 | 2025-10-07 10:00:00.000 |
| 4 | 4 | 1 | 4 | Average experience. | 3 | 2025-10-07 12:00:00.000 |
| 5 | 5 | 5 | 5 | Highly recommended. | 5 | 2025-10-08 16:00:00.000 |

| | reviewID | customerID | content | star_rating | review_dateTime |
|---|---|---|---|---|---|
| 1 | 1 | 1 | Very satisfied! | 5 | 2025-10-06 17:45:00.000 |
| 2 | 2 | 4 | Good but slow. | 4 | 2025-10-06 19:15:00.000 |
| 3 | 3 | 5 | Excellent quality. | 5 | 2025-10-07 10:30:00.000 |
| 4 | 4 | 1 | Could be better. | 3 | 2025-10-07 12:30:00.000 |
| 5 | 5 | 5 | Perfect service! | 5 | 2025-10-08 16:30:00.000 |

## Part D – SQL Queries & Outputs

### 1. Simple SELECT

```sql
-- 1. Show all customer names and their email address
SELECT firstName, lastName, email
FROM Users
WHERE role = 'Customer';
```

This query lists all users who are customers with their names and emails.

Results | Messages

| | firstName | lastName | email |
|---|---|---|---|
| 1 | John | Smith | john@gmail.com |
| 2 | Olivia | Taylor | olivia@gmail.com |
| 3 | Noah | Davis | noah@gmail.com |

### 2. JOIN

```sql
-- 2. List all orders with the customer's name and order type
SELECT u.firstName, u.lastName, o.orderID, o.type, o.status
FROM Orders o
JOIN Users u ON customerID = u.userID;
```

Shows which customers placed which orders and its status.

Results | Messages

| | firstName | lastName | orderID | type | status |
|---|---|---|---|---|---|
| 1 | John | Smith | 1 | Washing | Pending |
| 2 | Olivia | Taylor | 2 | Ironing | Pending |
| 3 | Noah | Davis | 3 | Dry Cleaning | Pending |
| 4 | John | Smith | 4 | Ironing | Pending |
| 5 | Noah | Davis | 5 | Washing & Ironing | Pending |

### 3. Aggregation

```sql
-- 3. Find total payment amout paid by all customers
SELECT SUM(amount) AS Total_Amount
FROM Payment
WHERE status = 'Paid';
```

Calculate the total revenue collected from paid payments.

Results | Messages

| | Total_Amount |
|---|---|
| 1 | 9700.00 |

### 4. GROUP BY / HAVING

```sql
-- 4. Find how many orders each customer placed
SELECT u.firstName, u.lastName, COUNT(o.orderID) AS TotalOrders
FROM Users u
JOIN Orders o ON u.userID = o.customerID
GROUP BY u.firstName, u.lastName
HAVING COUNT(o.orderID) > 1;
```

Groups orders by customers and show only those who made more than one order.

Results | Messages

| | firstName | lastName | TotalOrders |
|---|---|---|---|
| 1 | Noah | Davis | 2 |
| 2 | John | Smith | 2 |

## 5. Subquery

```sql
--5. show customer names who gave a 5 star feedback
SELECT firstName,lastName
FROM Users
WHERE userID IN (
SELECT customerID FROM Feedback WHERE star_rating = 5
);
```

Finds customers who have given the highest rating feedback.

Results | Messages

| | firstName | lastName |
|---|---|---|
| 1 | John | Smith |
| 2 | Noah | Davis |

## Part E – Stored Function

```sql
--Stored procedure/function
--Count pending payment for a customer
CREATE FUNCTION getPendingPaymentCount (@customer_id INT)
RETURNS INT
AS
BEGIN
    DECLARE @pendingCount INT;

    SELECT @pendingCount = COUNT(*)
    FROM Payment p
    INNER JOIN Orders o ON p.orderID = o.orderID
    WHERE o.customerID = @customer_id
      AND p.status = 'Pending';

    RETURN ISNULL(@pendingCount, 0);
END
GO

SELECT dbo.getPendingPaymentCount(1) AS PendingPayments;
SELECT dbo.getPendingPaymentCount(4) AS PendingPayments;
```

100 %

Results | Messages

| | PendingPayments |
|---|---|
| 1 | 0 |

| | PendingPayments |
|---|---|
| 1 | 1 |

## Part F – Trigger

**1.** The **core logic for recalculating the total order cost** is centralized within this T-SQL stored procedure

```sql

--###stored Procedure for recalculate the total cost of order a

GO

CREATE PROCEDURE recalculate_order_payment
    @p_orderID INT
AS
BEGIN
    -- Recalculate the total amount for the given orderID

    UPDATE p
    SET p.amount = c.CalculatedAmount
    FROM Payment p
    JOIN (
        -- Calculate the total cost: SUM(quantity * price)
        SELECT
            oi.orderID,
            ISNULL(SUM(oi.quantity * c.unit_price), 0.00) AS CalculatedAmount
        FROM OrderItem oi
        JOIN Cloth c ON oi.clothID = c.clothID
        WHERE oi.orderID = @p_orderID -- Filter by the input parameter
        GROUP BY oi.orderID
    ) AS c ON p.orderID = c.orderID
    WHERE p.orderID = @p_orderID;

END
GO

```

**2. AFTER UPDATE trigger** is defined to fire whenever an order item is modified

```sql
--###TRIGGER FOR AFTER UPDATE

GO

CREATE TRIGGER trg_recalculate_after_update
ON OrderItem
AFTER UPDATE
AS
BEGIN
    -- Only recalculate if the fields that affect the amount (quantity or clothID) have changed
    IF UPDATE(quantity) OR UPDATE(clothID)
    BEGIN
        DECLARE @OrderID INT;

        DECLARE OrderID_Cursor CURSOR FOR
        SELECT orderID FROM inserted; -- 'inserted' contains the NEW values

        OPEN OrderID_Cursor;
        FETCH NEXT FROM OrderID_Cursor INTO @OrderID;

        WHILE @@FETCH_STATUS = 0
        BEGIN
            EXEC recalculate_order_payment @OrderID;
            FETCH NEXT FROM OrderID_Cursor INTO @OrderID;
        END

        CLOSE OrderID_Cursor;
        DEALLOCATE OrderID_Cursor;
    END
END
GO
```

## 3. Demonstration Of Trigger

Before updating, the order's status (for Order_id :1)

```sql
-- Check the starting state for Order 1
SELECT
    oi.orderID,
    c.name AS cloth_name,
    oi.quantity,
    c.unit_price,
    (oi.quantity * c.unit_price) AS line_total,
    p.amount AS payment_amount
FROM OrderItem oi
JOIN Cloth c ON oi.clothID = c.clothID
JOIN Payment p ON oi.orderID = p.orderID
WHERE oi.orderID = 1;
```

Results Messages

| | orderID | cloth_name | quantity | unit_price | line_total | payment_amount |
|---|---|---|---|---|---|---|
| 1 | 1 | T-Shirt | 5 | 1500.00 | 7500.00 | 7500.00 |

After Updating:

```sql
UPDATE OrderItem
SET quantity = 6 -- Increase T-shirt quantity from 5 to 6
WHERE orderID = 1 AND clothID = 1;
-- Triggers trg_recalculate_after_update fires here.
```

Results Messages

| | orderID | cloth_name | quantity | unit_price | line_total | payment_amount |
|---|---|---|---|---|---|---|
| 1 | 1 | T-Shirt | 6 | 1500.00 | 9000.00 | 9000.00 |